Document number	P2657R1
Date	2022-11-14
Reply-to	Jarrad J. Waterloo <descender76 at gmail dot com>
Audience	Evolution Working Group (EWG)

C++ is the next C++

Table of contents

Changelog

R1

- aggregate multiple instances of the `static_analysis` attribute into the `inclusions` and `exclusions` members of the attribute
- added `std::const_pointer_cast` and `std::reinterpret_pointer_cast` to the No unsafe casts section
- added the Use ranges section
- use_function_ref is a subset of `safer` instead of `modern` because it reduces void* usage
- added the Tooling Opportunities section
- added the Reserved Behavior section

- removed the `How do we configure future analyzers` section from the Frequently Asked Questions as it is impossible to configure individually on the aggregate/grouped analyzers `safer` and `modern`, also consolidating using the attribute multiple times into one attribute
- added the How does this relate to p2687r0: Design Alternatives for Type-and-Resource Safe C++? section to the Frequently Asked Questions section
- added the Acknowledgments section

Abstract

Programmer's, Businesses and Government(s) want C++ to be safer and simpler. This has led some `C++` programmers to create new programming languages or preprocessors, which again is a new language. This paper discusses using static analysis to make the `C++` language itself safer and simpler.

Motivating Examples

Following is a wishlist. Most are optional. While, they all would be of benefit. It all starts with a new module level attribute that would preferably be applied once in the `primary module interface unit` and would automatically apply to it and all `module implementation unit` (s). It could also be applied to a `module implementation unit` but that would generally be less useful. However, it might aid in *gradual migration*.

```
export module some_module_name [[static_analysis(inclusions{"", "", ""}, exclusion
// or
module some_module_name [[static_analysis(inclusions{"", "", ""}, exclusions{"", "
```

- It would be ideal if the `inclusions` and `exclusions` members of the `static_analysis` attribute could be passed as either an environment variable and/or command line argument to compilers so it could be used by pipelines to assert the degree of conformance to the defined static analyzer without actually changing the source.
- It would be ideal if compilers could standardize the environment variable name or command line argument name in order to ease tooling.
- It would be ideal if compilers could produce a machine readable report in JSON, YAML or something else so that pipelines could more easily consume the results.
- It would be ideal if compilers could standardize the machine readable report.

The names of the `inclusions` and `exclusions` are dotted. Unscoped or names that start with `std.`, `c++.`, `cpp.`, `cxx.` or `c.` are reserved for standardization.

This proposal wants to stardardize two overarching static analyzer names; `safer` and `modern`.

`[[static_analysis(inclusions{"safer"})]]`	The `safer` analyzer is for safety, primarily memory related. It is for those businesses and programmers who must conform to safety standards. The `safer` analyzer is a subset of `modern` analyzer.
`[[static_analysis(inclusions{"modern"})]]`	The `modern` analyzer goes beyond just memory and safety concerns. It can be thought of as bleeding edge. It is for those businesses and programmers who commit to safety and higher quality modern code. That is those who want to enjoy the full benefits that C++ has to offer.

Neither is concerned about formatting or nitpicking. Both static analyzers only produce errors. These are meant for programmers, businesses and governments in which safety takes precedence. They both represent +∞; an ever increasing commitment. When a new version of the standard is released and adds new sub static analyzers than everyone's code is broken, until their code is fixed. These sub static analyzers usually consist of features that have been mostly replaced with some other feature. It would be ideal if the errors produced not only say that the code is wrong but also provide a link to html page(s) maintained by the `C++` teaching group, the authors of the `C++ Core Guidelines` [1] and compiler specific errors. These pages should provide example(s) of what is being replaced and by what was it replaced. Mentioning the version of the `C++` standard would also be helpful.

All modules can be used even if they don't use the `static_analysis` attribute as this allows *gradual adoption*.

The resolved list of static analyzers that will run is derived by first taking the union of all of the included analyzers including their component analyzers and removing from that union the union of all the excluded ananlyzers and their component analyzers.

resolved = (inclusion ∪ inclusion ∪ inclusion) - (exclusion ∪ exclusion ∪ exclusion)

This allows the programmer to exclude a few analyzers from a larger grouping instead of having to list out almost all the ever growing number of smaller analyzers that comprise the larger ones.

```
[[static_analysis(inclusions{"modern"}, exclusions{"use_ranges"})]];
```

What are the `safer` and `modern` analyzers composed of?

These overarching static analyzers are composed of multiple static analyzers which can be used individually to allow a degree of *gradual adoption*.

Use lvalue references

`[[static_analysis(inclusions{"use_lvalue_references"})]]`	`use_lvalue_referen` is a subset of `safer`.

- Any declaration of a pointer is an error.
- Calling any function that has parameters that take a pointer is an error unless the pointer type are “pointer to `const` character type” or “`const` pointer to `const` character type” and their arguments were string literals.
  - string literals are always safe having static storage duration
  - `std::string` and `std::string_view` must be creatable at compile time
- Function pointers and member function pointers can still be used.
  - Function pointers and member function pointers that declare pointers to non [member] function pointers produces an error.
- Lvalue references, &, can still be used.

WHY?

- A large portion of the C++ community have been programming without pointers for years. Some can go their whole career this way. This proposal just standardise existing practice.
- Modern `C++` has been advocated to programmers in other programming languages who complain about memory issues. This allows us to show them what we have been saying for decades.
- Over half of our memory related issues gets hashed away.
- Pointers have largely been replaced with the following:

lvalue references	1985: Cfront 1.0 [2]
STL	1992 [2:1]
`std::unique_ptr`, `std::shared_ptr`, `std::weak_ptr`, `std::reference_wrapper`, `std::make_shared`	C++11
`std::make_unique`	C++14
`std::string_view`, `std::optional`, `std::any`, `std::variant`	C++17
`std::make_shared` support arrays, `std::span`	C++20

The `C++ Core Guidelines` [1:1] identifies issues that this feature helps to mitigate.

- `P.4: Ideally, a program should be statically type safe` [3]
- `P.6: What cannot be checked at compile time should be checkable at run time` [4]
- `P.7: Catch run-time errors early` [5]
- `P.8: Don’t leak any resources` [6]
- `P.11: Encapsulate messy constructs, rather than spreading through the code` [7]
- `P.12: Use supporting tools as appropriate` [8]
- `P.13: Use support libraries as appropriate` [9]
- `I.4: Make interfaces precisely and strongly typed` [10]
- `I.11: Never transfer ownership by a raw pointer (T*) or reference (T&)` [11]
- `I.12: Declare a pointer that must not be null as not_null` [12]
- `I.13: Do not pass an array as a single pointer` [13]
- `I.23: Keep the number of function arguments low` [14]
- `F.7: For general use, take T* or T& arguments rather than smart pointers` [15]
- `F.15: Prefer simple and conventional ways of passing information` [16]
- `F.22: Use T* or owner<T*> to designate a single object` [17]
- `F.23: Use a not_null<T> to indicate that “null” is not a valid value` [18]
- `F.25: Use a zstring or a not_null<zstring> to designate a C-style string` [19]
- `F.26: Use a unique_ptr<T> to transfer ownership where a pointer is needed` [20]
- `F.27: Use a shared_ptr<T> to share ownership` [21]

- F.42: Return a T* to indicate a position (only) [22]
- F.43: Never (directly or indirectly) return a pointer or a reference to a local object [23]
- C.31: All resources acquired by a class must be released by the class’s destructor [24]
- C.32: If a class has a raw pointer (T*) or reference (T&), consider whether it might be owning [25]
- C.33: If a class has an owning pointer member, define a destructor [26]
- C.149: Use unique_ptr or shared_ptr to avoid forgetting to delete objects created using new [27]
- C.150: Use make_unique() to construct objects owned by unique_ptrs [28]
- C.151: Use make_shared() to construct objects owned by shared_ptrs [29]
- R.1: Manage resources automatically using resource handles and RAII (Resource Acquisition Is Initialization) [30]
- R.2: In interfaces, use raw pointers to denote individual objects (only) [31]
- R.3: A raw pointer (a T*) is non-owning [32]
- R.5: Prefer scoped objects, don’t heap-allocate unnecessarily [33]
- R.10: Avoid malloc() and free() [34]
- R.11: Avoid calling new and delete explicitly [35]
- R.12: Immediately give the result of an explicit resource allocation to a manager object [36]
- R.13: Perform at most one explicit resource allocation in a single expression statement [37]
- R.14: Avoid [] parameters, prefer span [38]
- R.15: Always overload matched allocation/deallocation pairs [39]
- R.20: Use unique_ptr or shared_ptr to represent ownership [40]
- R.22: Use make_shared() to make shared_ptrs [41]
- R.23: Use make_unique() to make unique_ptrs [42]
- ES.20: Always initialize an object [43]
- ES.24: Use a unique_ptr<T> to hold pointers [44]
- ES.42: Keep use of pointers simple and straightforward [45]
- ES.47: Use nullptr rather than 0 or NULL [46]
- ES.60: Avoid new and delete outside resource management functions [47]
- ES.61: Delete arrays using delete[] and non-arrays using delete [48]
- ES.65: Don’t dereference an invalid pointer [49]

- `E.13: Never throw while being the direct owner of an object` [50]
- `CPL.1: Prefer C++ to C` [51]

Gotchas

Usage of smart pointers

This static analyzer causes programmers to use 2 extra characters when using smart pointers, `->` vs `(*).`, since the overloaded `->` operator returns a pointer.

`smart_pointer->some_function();// bad`	`(*smart_pointer).some_function();//`

the main function and environment variables

A shim module is needed in order to transform main and env functions into a more C++ friendly functions. These have been asked for years.

1. `A Modern C++ Signature for main` [52]
2. `Desert Sessions: Improving hostile environment interactions` [53]

No unsafe casts

`[[static_analysis(inclusions{"no_unsafe_casts"})]]`	`no_unsafe_casts` is a subset of `safer`.

- Using `c` /core cast produces an error.
- Using `reinterpret_cast` produces an error.
- Using `const_cast` produces an error.
- Using `std::reinterpret_pointer_cast` produces an error.
- Using `std::const_pointer_cast` produces an error.

Why?

- `c` /core cast was replaced by `static_cast` and `dynamic_cast`.
- The `reinterpret_cast` is needed more for library authors than their users. For library users it usually just causes problems and questions. It is rarely used in daily `C++` when coding at a higher level.

- The `const_cast` is needed more for library authors than their users. It is a means for the programmer to lie to oneself. For library users it usually just causes problems and questions. It is rarely used in daily `C++` when coding at a higher level.

The `C++ Core Guidelines` [1:2] identifies issues that this feature helps to mitigate.

- `C.146: Use dynamic_cast where class hierarchy navigation is unavoidable` [54]
- `ES.48: Avoid casts` [55]
- `ES.49: If you must use a cast, use a named cast` [56]
- `ES.50: Don’t cast away const` [57]

No unions

`[[static_analysis(inclusions{"no_union"})]]`	`no_union` is a subset of `safer`.

- Using the `union` keyword produces an error.

It was replaced by `std::variant`, which is safer.

The `C++ Core Guidelines` [1:3] identifies issues that this feature helps to mitigate.

- `C.181: Avoid “naked” unions` [58]

No mutable

`[[static_analysis(inclusions{"no_mutable"})]]`	`no_mutable` is a subset of `safer`.

- Using the `mutable` keyword produces an error.

The programmer shall not lie to oneself. The `mutable` keyword violates the safety of `const` and is rarely used at a high level.

No new or delete

`[[static_analysis(inclusions{"no_new_delete"})]]`	`no_new_delete` is a subset of `safer`.

- Using the `new` and `delete` keywords to allocate and deallocate memory produces an error.

It was replaced by `std::make_unique` and `std::make_shared`, which are safer.

The `C++ Core Guidelines` [1:4] identifies issues that this feature helps to mitigate.

- `F.26: Use a unique_ptr<T> to transfer ownership where a pointer is needed` [20:1]
- `F.27: Use a shared_ptr<T> to share ownership` [21:1]
- `C.149: Use unique_ptr or shared_ptr to avoid forgetting to delete objects created using new` [27:1]
- `C.150: Use make_unique() to construct objects owned by unique_ptrs` [28:1]
- `C.151: Use make_shared() to construct objects owned by shared_ptrs` [29:1]
- `R.11: Avoid calling new and delete explicitly` [35:1]
- `R.20: Use unique_ptr or shared_ptr to represent ownership` [40:1]
- `R.22: Use make_shared() to make shared_ptrs` [41:1]
- `R.23: Use make_unique() to make unique_ptrs` [42:1]
- `ES.60: Avoid new and delete outside resource management functions` [47:1]
- `ES.61: Delete arrays using delete[] and non-arrays using delete` [48:1]

No volatile

`[[static_analysis(inclusions{"no_volatile"})]]`	`no_volatile` is a subset of `safer`.

- Using the `volatile` keyword produces an error.

The `volatile` keyword has nothing to do with concurrency. Use `std::atomic` or `std::mutex` instead.

The `C++ Core Guidelines` [1:5] identifies issues that this feature helps to mitigate.

- `CP.8: Don't try to use volatile for synchronization` [59]

No `C` style variadic functions

`[[static_analysis(inclusions{"no_c_style_variadic_functions"})]]`	`no_c_style_` is a subset o

- Declaring a `C` style variadic function produces an error.
- Calling a `C` style variadic function produces an error.
- Using the `va_start`, `va_arg`, `va_copy`, `va_end` or `va_list` functions produces errors.

`C` style variadic functions has been replaced by overloading, templates and variadic template functions.

The `C++ Core Guidelines` [1:6] identifies issues that this feature helps to mitigate.

- `F.55: Don't use va_arg arguments` [60]
- `ES.34: Don't define a (C-style) variadic function` [61]

No deprecated

`[[static_analysis(inclusions{"no_deprecated"})]]`	`no_deprecated` is a subset of `modern`.

- Using anything that has the deprecated attribute on it produces an error.

Deprecated functionality is not modern.

Use `std::array`

`[[static_analysis(inclusions{"use_std_array"})]]`	`use_std_array` is a subset of `modern`.

- Declaring a `C` style/core `C++` array variable, whether locally or in a class, produces an error.
- It is okay to use array literals when initializing `std::array` and other collections.

Use `std::array` instead of `C` style/core `C++` array.

Use ranges

`[[static_analysis(inclusions{"use_ranges"})]]`	`use_ranges` is a subset of `modern`.

Using any iterator based algorithm that has been replaced with a range based algorithm produces an error informing the programmer to use the range based algorithm instead.

- Using `std::all_of` produces an error.
- Using `std::any_of` produces an error.
- Using `std::none_of` produces an error.
- Using `std::for_each` produces an error.
- Using `std::for_each_n` produces an error.
- Using `std::count` produces an error.
- Using `std::count_if` produces an error.
- Using `std::mismatch` produces an error.
- Using `std::find` produces an error.
- Using `std::find_if` produces an error.
- Using `std::find_if_not` produces an error.
- Using `std::find_end` produces an error.
- Using `std::find_first_of` produces an error.
- Using `std::adjacent_find` produces an error.
- Using `std::search` produces an error.
- Using `std::search_n` produces an error.
- Using `std::copy` produces an error.
- Using `std::copy_if` produces an error.
- Using `std::copy_n` produces an error.
- Using `std::copy_backward` produces an error.
- Using `std::move` produces an error.
- Using `std::move_backward` produces an error.
- Using `std::fill` produces an error.
- Using `std::fill_n` produces an error.
- Using `std::transform` produces an error.
- Using `std::generate` produces an error.

- Using `std::generate_n` produces an error.
- Using `std::remove` produces an error.
- Using `std::remove_if` produces an error.
- Using `std::remove_copy` produces an error.
- Using `std::remove_copy_if` produces an error.
- Using `std::replace` produces an error.
- Using `std::replace_if` produces an error.
- Using `std::replace_copy` produces an error.
- Using `std::replace_copy_if` produces an error.
- Using `std::swap_ranges` produces an error.
- Using `std::reverse` produces an error.
- Using `std::reverse_copy` produces an error.
- Using `std::rotate` produces an error.
- Using `std::rotate_copy` produces an error.
- Using `std::shift_left` produces an error.
- Using `std::shift_right` produces an error.
- Using `std::shuffle` produces an error.
- Using `std::unique` produces an error.
- Using `std::unique_copy` produces an error.
- Using `std::is_partitioned` produces an error.
- Using `std::partition` produces an error.
- Using `std::partition_copy` produces an error.
- Using `std::stable_partition` produces an error.
- Using `std::partition_point` produces an error.
- Using `std::is_sorted` produces an error.
- Using `std::is_sorted_until` produces an error.
- Using `std::sort` produces an error.
- Using `std::partial_sort` produces an error.
- Using `std::partial_sort_copy` produces an error.
- Using `std::stable_sort` produces an error.
- Using `std::nth_element` produces an error.
- Using `std::lower_bound` produces an error.
- Using `std::upper_bound` produces an error.
- Using `std::binary_search` produces an error.

- Using `std::equal_range` produces an error.
- Using `std::merge` produces an error.
- Using `std::includes` produces an error.
- Using `std::set_difference` produces an error.
- Using `std::set_intersection` produces an error.
- Using `std::set_symmetri_difference` produces an error.
- Using `std::set_union` produces an error.
- Using `std::is_heap` produces an error.
- Using `std::is_heap_until` produces an error.
- Using `std::make_heap` produces an error.
- Using `std::push_heap` produces an error.
- Using `std::pop_heap` produces an error.
- Using `std::sort_heap` produces an error.
- Using `std::max` produces an error.
- Using `std::max_element` produces an error.
- Using `std::min` produces an error.
- Using `std::min_element` produces an error.
- Using `std::minmax` produces an error.
- Using `std::minmax_element` produces an error.
- Using `std::clamp` produces an error.
- Using `std::equal` produces an error.
- Using `std::lexicographical_compare` produces an error.
- Using `std::is_permutation` produces an error.
- Using `std::next_permutation` produces an error.
- Using `std::prev_permutation` produces an error.
- Using `std::iota` produces an error.
- Using `std::uninitialized_copy` produces an error.
- Using `std::uninitialized_copy_n` produces an error.
- Using `std::uninitialized_fill` produces an error.
- Using `std::uninitialized_fill_n` produces an error.
- Using `std::uninitialized_move` produces an error.
- Using `std::uninitialized_move_n` produces an error.
- Using `std::uninitialized_default_construct` produces an error.
- Using `std::uninitialized_default_construct_n` produces an error.

- Using `std::uninitialized_value_construct` produces an error.
- Using `std::uninitialized_value_construct_n` produces an error.
- Using `std::destroy` produces an error.
- Using `std::destroy_n` produces an error.
- Using `std::destroy_at` produces an error.
- Using `std::construct_at` produces an error.

What may `safer` and `modern` analyzers be composed of in the future?

No include

`[[static_analysis(inclusions{"no_include"})]]`	`no_include` is a subset of `modern`.

The preprocessor directive `#include` has been replaced with `import`. Don't add the static analyzer until `#embed` is added.

NOTE: This may be impossible to implement as preprocessing occurs before compilation.

No goto

`[[static_analysis(inclusions{"no_goto"})]]`	`no_goto` is a subset of `modern`.

- Using the `goto` keyword produces an error.

Don't add until `break` and `continue` to a label is added. Also a really easy to use finite state machine library may be needed.

The `C++ Core Guidelines` [1:7] identifies issues that this feature helps to mitigate.

- `ES.76: Avoid goto` [62]

`Use std::function_ref`

`[[static_analysis(inclusions{"use_function_ref"})]]`	`use_function_ref` is a subset of `safer`.

- Declaring a `C` style/core `C++` function pointer, whether locally or in a class, produces an error.
- Declaring a `C` style/core `C++` member function pointer, whether locally or in a class, produces an error.
- It is okay to use [member] function pointer literals when initializing `std::function_ref` and others.

Use `std::function_ref` instead of `C` style/core `C++` [member] function pointers. `std::function_ref` can bind to stateful and stateless, free and member functions. It saves programmers from having to include a `void*` state parameter in their function pointer types and it also saves from having to include `void*` state parameter along side the function pointer type in each function where the function pointer type is used in function declarations. Neither of which could be performed with the `"use_lvalue_references"` static analyzer.

NOTE:

- This can't be performed until `nontype_t` [63] `std::function_ref` [64] gets standardized.

Tooling Opportunities

Automated Code Reviews

In the Motivating Examples section there were two specific wishlist items.

- *It would be ideal if compilers could produce a machine readable report in JSON, YAML or something else so that pipelines could more easily consume the results.*
- *It would be ideal if compilers could standardize the machine readable report.*

With these capabilities, a report could be created during a distributed version control system's pull/merge request. The report could be compared to the report of the destination of the request. If the changed code is not better than the existing code than the request can be automatically rejected. This would result in an adaptation of the boy scout rule.

Leave the code cleaner than you found it.

Consequently, this results in the creation of a programmer incline. With each checkin, the code gets better. The incline can even be adjusted by requiring how much better one must leave the code.

Reserved Behavior

The `static_analysis` attribute can only, for now, be used on either `primary module interface unit` or `module implementation unit` but not both at the same time. Enabling it in both would require a discussion of how these analyzers should combine. Which one would take precedence? It would need to be part of a larger discussion of whether the `static_analysis` attribute could be applied at the namespace, class, function or control block levels. This proposal is focused on the module level as current static analyzers on the market is more focused on the translation unit level rather than on a per line basis. As such, this proposal could be adopted faster, yet, leaving room for improvements, once static analyzers improve in their precision.

Summary

By adding static analysis to the `C++` language we can make the language safer and easier to teach because we can restrict how much of the language we use. Human readable errors and references turns the compiler into a teacher freeing human teachers to focus on what the compiler doesn't handle.

Frequently Asked Questions

Shouldn't these be warnings instead of errors?

NO, otherwise we'll be stuck with what we just have. `C++` compilers produces plenty of warnings. `C++` static analyzers produces plenty of warnings. However, when some one talks about creating a new language, then old language syntax becomes invalid i.e. errors. This is for programmers. Programmers and businesses rarely upgrade their code unless they are forced to. Businesses and Government(s) want errors, as well, in order to ensure code quality and the assurance that bad code doesn't exist anywhere in the module. This is also important from a language standpoint because we are essentially pruning; somewhat. Keep in mind that all of these pruned features still have use now. In the future, more constructs will be built upon these pruned features. This is why they need to be part of the language, just not a part of everyday usage of the language.

Why at the module level? Why not safe and unsafe blocks?

Programmers and businesses rarely upgrade their code unless they are forced to. New programmers need training wheels and some of us older programmers like them too. Due to the proliferation of government regulations and oversight, businesses have acquired `software composition analysis` services and tools. These services map security errors to specific versions of modules; specifically programming artifacts such as executables and libraries. As such, businesses want to know if a module is reasonably safe.

You must really hate pointers?

Actually, I love `C`, `C++` and pointers.

- I recognize that most of the time, when I code, that I don't need them.
- I recognize that **past** fundamental `C++` libraries use pointers but the users of those libraries don't need them.
- I recognize that **present** fundamental libraries such `function_ref` uses `void*` for type erasure but the users of `function_ref`, most of the time, won't need it.
- I recognize that **future** fundamental libraries such as dynamic polymorphic traits also need pointers for type erasure but they don't expect their users to fidget with raw pointers.
- I also recognize that 1 programmer writes a library but hundreds use the library without needing the same parts of C++ used in its creation.
- Pointers are simple and easy for memory mapped hardware but many C++ programmers don't operate at this level.
- A few will create an OS [driver] but thousands will use it.

The fact is pointers, unsafe casts, `union`, `mutable` and `goto` are the engine of C++ change. As such it would be foolish to remove them but it is also unrealistic for users/drivers of a vehicle to have to drive with nothing between them and the engine, without listening to them clamor for interior finishing.

C++ can't standardize specific static analyzers

- Can't `C++` provide the `static_analysis` attribute so that static analyzers can be called?
- Can't `C++` reserve unscoped or names that start with `std.`, `c++.`, `cpp.`, `cxx.` or `c.` are for future standardization?
- Can't `C++` reserve the names of static analyzers in the reserved `C++` static analyzer namespace?
- Can't `C++` **recommend** these reserved static analyzers and leave it to the compiler writers to appease their users that clamor for them?

Do you fear that this could create a “subset of C++” that “could split the user community and cause acrimony”? [65]

First of all, let’s consider the quotes of Bjarne Stroustrup that this question are based upon.

> *“being defined by an ‘industry consortium.’ I am not in favor of language subsets or dialects. I am especially not fond of subsets that cannot support the standard library so that the users of that subset must invent their own incompatible foundation libraries. I fear that a defined subset of C++ could split the user community and cause acrimony”* [65:1]

Does this paper create a subset? YES. Like it or not `C++` already have a couple of subsets; some positive, some quasi. `Freestanding` is a subset for low level programming. This proposal primarily focus on high level programming but there is nothing preventing the creation of `[[static_analysis(inclusions{"freestanding"})]]` which enforces `freestanding`. The `C++` value categories has to some degree fractured the community into a clergy class that thoroughly understand its intracacies and a leity class that gleefully uses it.

Does this paper split the user community? YES and NO. It splits code into safer vs. less safe, high level vs. low level. However, this is performed at the module level, allowing the same programmer to decide what falls on either side of the fence. This would not be performed by an industry consortium but rather the standard. Safer modules can be used by less safe modules. Less safe modules can partly be used by safer modules, such as with the standard module. This latter impact is already minimalized because the standard frequently write their library code in `C++` fashion instead of a `C` fashion.

> ***“Are there any features you’d like to remove from C++?”*** [66]
>
> *Not really. People who ask this kind of question usually think of one of the major features such as multiple inheritance, exceptions, templates, or run-time type identification. C++ would be incomplete without those. I have reviewed their design over the years, and together with the standards committee I have improved some of their details, but none could be removed without doing damage.* [66:1]
>
> *Most of the features I dislike from a language-design perspective (e.g., the declarator syntax and array decay) are part of the C subset of C++ and couldn’t be removed without doing harm to programmers working under real-world conditions. C++'s C compatibility was a key language design decision rather than a marketing gimmick. Compatibility has been difficult to achieve and maintain, but real benefits to real programmers resulted, and still result today. By now, C++ has features that allow a programmer to refrain from using the most troublesome C features. For example, standard library containers such as vector, list, map, and string can be used to avoid most tricky low-level pointer manipulation.* [66:2]

The beauty of this proposal is it does not and it does remove features from C++. Like the standard library, it allows programmers to refrain from using the most troublesome `c` and `c++` features.

> “Within C++, there is a much smaller and cleaner language struggling to get out” [67]

Both making things smaller and cleaner requires removing something. When creating a new language, removing things happens extensively at the beginning but, frequently, features have to be added back in, when programmers clamor for them. This paper cleans up a programmers use of the `c++` language, meaning less `c++` has to be taught immediately, thus making things simpler. As a programmer matures, features can be gradually added to their repertoire, just as it was added to ours. After all, isn’t `c++` larger now, than when we started programming in `c++`.

How does this relate to p2687r0: Design Alternatives for Type-and-Resource Safe C++?

This proposal and the “Design Alternatives for Type-and-Resource Safe C++” [68] proposal both recommend that static analysis be used and brought into the language instead of inventing a whole new language. Both tackles problems in its own way. Either proposal could be enhanced to do what the other proposal does. The question is what are these differences and should these be given some attention.

Different audiences

This proposal might appeal more to non voting, newer programmers working on smaller, newer code bases. The `p2687r0` proposal appeals more to voting, older programmers working on larger, older code bases. There are also differences in the sizes of these two audiences. This proposal would have the larger audience as it appeals to those who want a subset of language and library features. There are also differences in the level of coding. This proposal favors high level, abstraction heavy coding. The `p2687r0` proposal appeals more to lower level, closer to hardware coding. Again both proposals fixes safety issues and either audience just wants more safety, **sooner, rather than later**.

Are there any elements of this proposal that would still appeal to lower level coders? New code does get developed in older code bases. The question is do you want programmers to keep writing their code the old way for the sake of a foolish consistency! So this proposal is of use to lower level programmers. With the `p2687r0` proposal, a lot of time is spent analyzing and documenting with attributes the intention of pointers at each point of use in the code. No rewrite is being performed and more information is being provided to resolve ambiguity for the benefit of the static analyzer. The cost of this programmer analysis and attributing can be most of the cost of a rewrite, so why not just rewrite it incrementally in `safer modern C++` ! This proposal helps even with this. From my experience with lower level code, I tend to have a few files of the majority that does the work with memory mapped files or that call `C` API's but once I have my wrappers, the remainer of my code is very high level and abstract. So in this regard, this proposal is of benefit. `C++20` modules are still `new` even to existing code bases especially since tool chains are still being developed and their are still many unanswered questions. Since there will need to be incremental refactoring to use modules in older code bases, why not take advantage of this proposal's module level attribute to take advantage of more refactoring.

Different scopes

This proposal has fewer features than `p2687r0` . For instance, it currently only works at the module level. This is similar to where many static analyzers run, at the translation unit level. While this proposal has far fewer features, it is smaller, simpler and easier to implement. This could mean the difference in getting some subset of safety in the `C++26/C++29` timeframe instead of the `C++29/C++32` timeframe. The additional features could be added incrementally.

Different solutions

The `p2687r0` proposal tackles problems head on. Bravo! This proposal is about avoiding problems, all together, by using existing language and library features, that we have had for years, if not decades, but just needed the option of enforcement. Both, I know, have merit. Some problems are left by this proposal deliberately to other proposals.

For instance, on the subject of dangling, it is best to fix more of this in the language instead of the analyzer. With the following two proposals, the dangling mountain could be shrunk to a mole-hill or ant-hill.

- `implicit constant initialization` [69]
- `temporary storage class specifiers` [70]

Further adding the paper that those two were based on would further shrink dangling to a few grains of sand on a sea shore of code.

- `Bind Returned/Initialized Objects to Lifetime of Parameters` [71]
- `[RFC] Lifetime annotations for C++` [72]

On the subject of type safety, this paper agrees with the `p2687r0` proposal on the usage of SELL, Semantically Enhanced Language Libraries. Currently, there is work ongoing in the standardization process to provide a standard units library which would go a long ways for type safety. Currently, there is work ongoing in the standardization process to provide a standard graph library which would go a long ways for the more extreme memory safety. Still unproposed but still needed are strongly typed alias library or language features for safer `int` (s). Enhancements to existing fundamental types in `C++` could include validation and tag classes in order to make those types safer.

In short, this proposal is, in some ways, a subset of the `p2687r0` proposal. Combined with other proposals, they beat the most notorious safety problems into an acceptable level of safety to many.

Acknowledgments

Thanks to Vladimir Smirnov for providing very valuable feedback on this proposal.

References

1. https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines
(https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines) ↩ ↩ ↩ ↩ ↩ ↩ ↩ ↩

2. https://en.cppreference.com/w/cpp/language/history
(https://en.cppreference.com/w/cpp/language/history) ↩ ↩

3. https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#p4-ideally-a-program-should-be-statically-type-safe (https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#p4-ideally-a-program-should-be-statically-type-safe) ↩

4. https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#p6-what-cannot-be-checked-at-compile-time-should-be-checkable-at-run-time (https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#p6-what-cannot-be-checked-at-compile-time-should-be-checkable-at-run-time) ↩

5. https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#p7-catch-run-time-errors-early (https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#p7-catch-run-time-errors-early) ↩

6. https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#p8-dont-leak-any-resources (https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#p8-dont-leak-any-resources) ↩

7. https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#p11-encapsulate-messy-constructs-rather-than-spreading-through-the-code (https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#p11-encapsulate-messy-constructs-rather-than-spreading-through-the-code) ↩

8. https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#p12-use-supporting-tools-as-appropriate (https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#p12-use-supporting-tools-as-appropriate) ↩

9. https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#p13-use-support-libraries-as-appropriate (https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#p13-use-support-libraries-as-appropriate) ↩

10. https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#i4-make-interfaces-precisely-and-strongly-typed (https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#i4-make-interfaces-precisely-and-strongly-typed) ↩

11. https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#i11-never-transfer-ownership-by-a-raw-pointer-t-or-reference-t (https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#i11-never-transfer-ownership-by-a-raw-pointer-t-or-reference-t) ↩

12. https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#i12-declare-a-pointer-that-must-not-be-null-as-not_null (https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#i12-declare-a-pointer-that-must-not-be-null-as-not_null) ↩

13. https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#i13-do-not-pass-an-array-as-a-single-pointer (https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#i13-do-not-pass-an-array-as-a-single-pointer) ↩

14. https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#i23-keep-the-number-of-function-arguments-low (https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#i23-keep-the-number-of-function-arguments-low) ↩

15. https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#f7-for-general-use-take-t-or-t-arguments-rather-than-smart-pointers (https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#f7-for-general-use-take-t-or-t-arguments-rather-than-smart-pointers) ↩

16. https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#f15-prefer-simple-and-conventional-ways-of-passing-information (https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#f15-prefer-simple-and-conventional-ways-of-passing-information) ↩

17. https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#f22-use-t-or-ownert-to-designate-a-single-object (https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#f22-use-t-or-ownert-to-designate-a-single-object) ↩

18. https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#f23-use-a-not_nullt-to-indicate-that-null-is-not-a-valid-value (https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#f23-use-a-not_nullt-to-indicate-that-null-is-not-a-valid-value) ↩

19. https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#f25-use-a-zstring-or-a-not_nullzstring-to-designate-a-c-style-string (https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#f25-use-a-zstring-or-a-not_nullzstring-to-designate-a-c-style-string) ↩

20. https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#f26-use-a-unique_ptrt-to-transfer-ownership-where-a-pointer-is-needed (https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#f26-use-a-unique_ptrt-to-transfer-ownership-where-a-pointer-is-needed) ↩ ↩

21. https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#f27-use-a-shared_ptrt-to-share-ownership (https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#f27-use-a-shared_ptrt-to-share-ownership) ↩ ↩

22. https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#f42-return-a-t-to-indicate-a-position-only (https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#f42-return-a-t-to-indicate-a-position-only) ↩

23. https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#f43-never-directly-or-indirectly-return-a-pointer-or-a-reference-to-a-local-object (https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#f43-never-directly-or-indirectly-return-a-pointer-or-a-reference-to-a-local-object) ↩

24. https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#c31-all-resources-acquired-by-a-class-must-be-released-by-the-classs-destructor (https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#c31-all-resources-acquired-by-a-class-must-be-released-by-the-classs-destructor) ↩

25. https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#c32-if-a-class-has-a-raw-pointer-t-or-reference-t-consider-whether-it-might-be-owning (https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#c32-if-a-class-has-a-raw-pointer-t-or-reference-t-consider-whether-it-might-be-owning) ↩

26. https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#c33-if-a-class-has-an-owning-pointer-member-define-a-destructor (https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#c33-if-a-class-has-an-owning-pointer-member-define-a-destructor) ↩

27. https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#c149-use-unique_ptr-or-shared_ptr-to-avoid-forgetting-to-delete-objects-created-using-new (https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#c149-use-unique_ptr-or-shared_ptr-to-avoid-forgetting-

to-delete-objects-created-using-new) ↩ ↩

28. https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#c150-use-make_unique-to-construct-objects-owned-by-unique_ptrs (https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#c150-use-make_unique-to-construct-objects-owned-by-unique_ptrs) ↩ ↩

29. https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#c151-use-make_shared-to-construct-objects-owned-by-shared_ptrs (https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#c151-use-make_shared-to-construct-objects-owned-by-shared_ptrs) ↩ ↩

30. https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#r1-manage-resources-automatically-using-resource-handles-and-raii-resource-acquisition-is-initialization (https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#r1-manage-resources-automatically-using-resource-handles-and-raii-resource-acquisition-is-initialization) ↩

31. https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#r2-in-interfaces-use-raw-pointers-to-denote-individual-objects-only (https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#r2-in-interfaces-use-raw-pointers-to-denote-individual-objects-only) ↩

32. https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#r3-a-raw-pointer-a-t-is-non-owning (https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#r3-a-raw-pointer-a-t-is-non-owning) ↩

33. https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#r5-prefer-scoped-objects-dont-heap-allocate-unnecessarily (https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#r5-prefer-scoped-objects-dont-heap-allocate-unnecessarily) ↩

34. https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#r10-avoid-malloc-and-free (https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#r10-avoid-malloc-and-free) ↩

35. https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#r11-avoid-calling-new-and-delete-explicitly (https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#r11-avoid-calling-new-and-delete-explicitly) ↩ ↩

36. https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#r12-immediately-give-the-result-of-an-explicit-resource-allocation-to-a-manager-object (https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#r12-immediately-give-the-result-of-an-explicit-resource-allocation-to-a-manager-object) ↩

37. https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#r13-perform-at-most-one-explicit-resource-allocation-in-a-single-expression-statement (https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#r13-perform-at-most-one-explicit-resource-allocation-in-a-single-expression-statement) ↩

38. https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#r14-avoid--parameters-prefer-span (https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#r14-avoid--parameters-prefer-span) ↩

39. https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#r15-always-overload-matched-allocationdeallocation-pairs (https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#r15-always-overload-matched-allocationdeallocation-pairs) ↩

40. https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#r20-use-unique_ptr-or-shared_ptr-to-represent-ownership (https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#r20-use-unique_ptr-or-shared_ptr-to-represent-ownership) ↩ ↩

41. https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#r22-use-make_shared-to-make-shared_ptrs (https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#r22-use-make_shared-to-make-shared_ptrs) ↩ ↩

42. https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#r23-use-make_unique-to-make-unique_ptrs (https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#r23-use-make_unique-to-make-unique_ptrs) ↩ ↩

43. https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#es20-always-initialize-an-object (https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#es20-always-initialize-an-object) ↩

44. https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#es24-use-a-unique_ptrt-to-hold-pointers (https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#es24-use-a-unique_ptrt-to-hold-pointers) ↩

45. https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#es42-keep-use-of-pointers-simple-and-straightforward (https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#es42-keep-use-of-pointers-simple-and-straightforward) ↩

46. https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#es47-use-nullptr-rather-than-0-or-null (https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#es47-use-nullptr-rather-than-0-or-null) ↩

47. https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#es60-avoid-new-and-delete-outside-resource-management-functions (https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#es60-avoid-new-and-delete-outside-resource-management-functions) ↩ ↩

48. https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#es61-delete-arrays-using-delete-and-non-arrays-using-delete (https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#es61-delete-arrays-using-delete-and-non-arrays-using-delete) ↩ ↩

49. https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#es65-dont-dereference-an-invalid-pointer (https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#es65-dont-dereference-an-invalid-pointer) ↩

50. https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#e13-never-throw-while-being-the-direct-owner-of-an-object (https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#e13-never-throw-while-being-the-direct-owner-of-an-object) ↩

51. https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#cpl1-prefer-c-to-c (https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#cpl1-prefer-c-to-c) ↩

52. https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/p0781r0.html (https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/p0781r0.html) ↩

53. https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p1275r0.html (https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p1275r0.html) ↩

54. https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#c146-use-dynamic_cast-where-class-hierarchy-navigation-is-unavoidable (https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#c146-use-dynamic_cast-where-class-hierarchy-navigation-is-unavoidable) ↩

55. https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#es48-avoid-casts (https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#es48-avoid-casts) ↩

56. https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#es49-if-you-must-use-a-cast-use-a-named-cast (https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#es49-if-you-must-use-a-cast-use-a-named-cast) ↩

57. https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#es50-dont-cast-away-const (https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#es50-dont-cast-away-const) ↩

58. https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#c181-avoid-naked-unions (https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#c181-avoid-naked-unions) ↩

59. https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#cp8-dont-try-to-use-volatile-for-synchronization (https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#cp8-dont-try-to-use-volatile-for-synchronization) ↩

60. https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#f55-dont-use-va_arg-arguments (https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#f55-dont-use-va_arg-arguments) ↩

61. https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#-es34-dont-define-a-c-style-variadic-function (https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#-es34-dont-define-a-c-style-variadic-function) ↩

62. https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#es76-avoid-goto (https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#es76-avoid-goto) ↩

63. https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p2472r3.html (https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p2472r3.html) ↩

64. https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p0792r11.html (https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p0792r11.html) ↩

65. https://www.stroustrup.com/bs_faq.html#EC++ (https://www.stroustrup.com/bs_faq.html#EC++) ↩ ↩

66. https://www.stroustrup.com/bs_faq.html#remove-from-C++ (https://www.stroustrup.com/bs_faq.html#remove-from-C++) ↩ ↩ ↩

67. https://www.stroustrup.com/quotes.html (https://www.stroustrup.com/quotes.html) ↩

68. https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p2687r0.pdf (https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p2687r0.pdf) ↩

69. https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p2623r2.html (https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p2623r2.html) ↩

70. https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p2658r0.html (https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p2658r0.html) ↩

71. http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p0936r0.pdf (http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p0936r0.pdf) ↩

72. https://discourse.llvm.org/t/rfc-lifetime-annotations-for-c/61377 (https://discourse.llvm.org/t/rfc-lifetime-annotations-for-c/61377) ↩